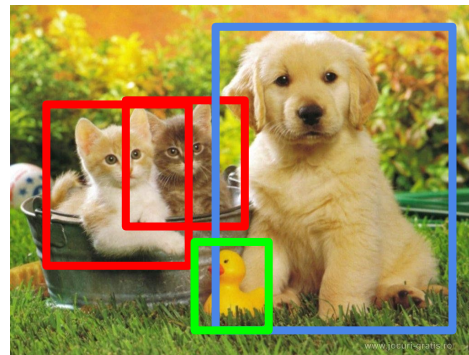


Tasks

- Task 1: Object detection
 - **Task 1.1: Off-the-shelf**
 - Task 1.3: Fine-tune to your data
 - Task 1.4: K-Fold Cross-validation
- Task 2: Object tracking



Task 1.1: Object Detection: Off-the-Shelf (Team 5)

□ Ground Truth
□ Predictions

Objective

Per-frame vehicle detection baseline.

Setup & Evaluation

- Benchmark of models pre-trained on MS COCO, filtering exclusively for Class ID 3 ('car').
- We aim to detect both moving and stationary vehicles in the sequence.
- Heavily occluded vehicles are omitted from Ground Truth (GT) to maintain parity with Week 1.
- Primary Metric: mAP@0.5 (evaluated per COCO standards).

Models

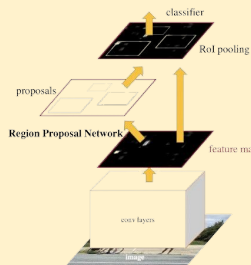
Faster R-CNN[1]

Why? Industry standard for high-precision two-stage detection.

Framework: PyTorch (Torchvision)

Backbone: ResNet50-FPN[2]

Complexity: 41.7M parameters.



Implemented via
PyTorch

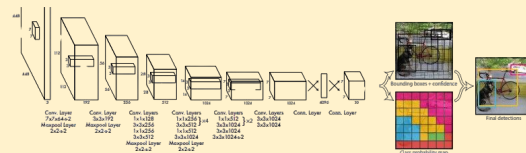
YOLOv8[3] (One-Stage SOTA) Medium/Nano

Why? State-of-the-art balance between accuracy and inference latency.

Framework: Ultralytics

Type: End-to-end one-stage detector.

Complexity: 25.9M (Medium) – 3.2M (Nano)



Implemented via
Ultralytics

Ran with...

NVIDIA
Nvidia GeForce RTX 3090



The cars behind the prediction/ground truth are ignored in order to compute the metrics.

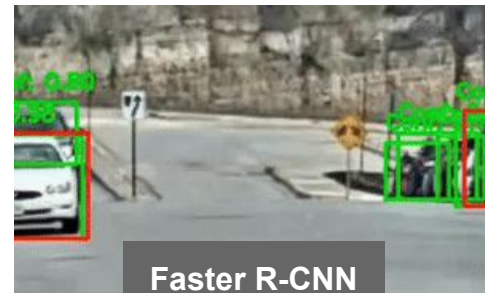
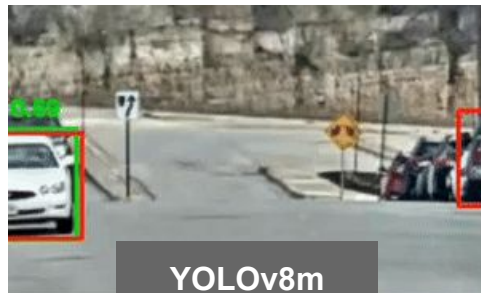
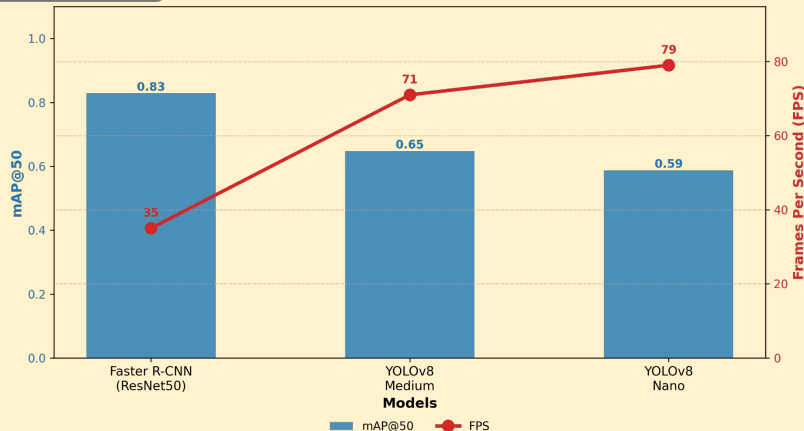
[1] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 91–99.
[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).
[3] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 (Version 8.0.0) [Software]. Available at <https://github.com/ultralytics/ultralytics>.

Task 1.1: Object Detection: Off-the-Shelf (Team 5)

□ Ground Truth
□ Predictions

Results

Task 1.1: Accuracy (mAP) vs. Speed (FPS) Trade-off



Visual Inspection



Qualitative analysis confirms that Faster **R-CNN** handles background and distant objects significantly better. This is largely due to the Feature Pyramid Network (FPN) and localized region proposals, which provide **better scale-invariance compared to YOLO's** global grid approach.

Analysis

- Quantitative results indicate that while **Faster R-CNN** has **higher inference latency**, it achieves a significantly **superior mAP@0.5** of **0.83**, outperforming YOLOv8-m by a margin of 0.18.
- In terms of throughput, **YOLOv8** variants exhibit **high-speed processing**, exceeding 70 FPS, effectively doubling the frame rate of Faster R-CNN.

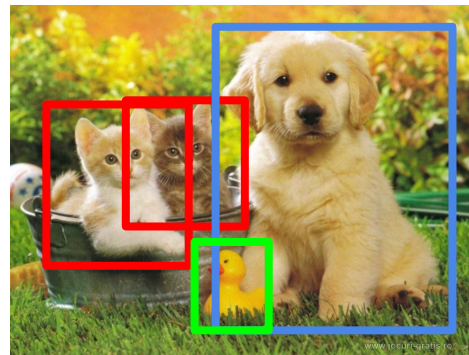


Winner Model

In this project we decided to **prioritize the performance** over the speed. Plus, R-CNN already works faster than the 20 FPS of the AICity video. **We select Faster R-CNN** as the baseline, favoring precision and small-object detection capabilities over computational speed.

Tasks

- Task 1: Object detection
 - Task 1.1: Off-the-shelf
 - **Task 1.2: Fine-tune to your data**
 - Task 1.3: K-Fold Cross-validation
- Task 2: Object tracking



Task 1.2: Object Detection: Fine-tune (Team 5)

Methodology

Load pre-trained
R-CNN



Freeze backbone

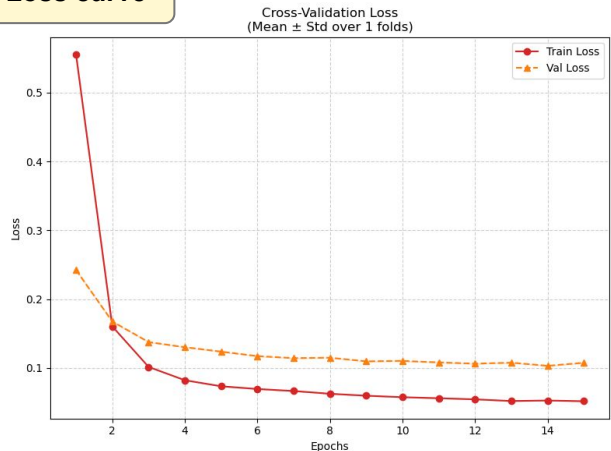


Train detection head
on first 25% frames



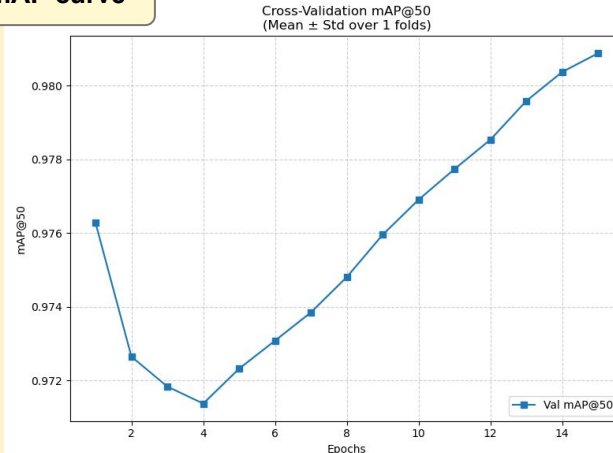
Evaluate on last 75%
frames

Loss curve



Convergence achieved around epoch 10.
The model overfits a bit the data,
Val Loss > Train Loss.

mAP curve



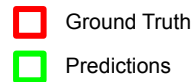
Initial **mAP drop** suggests a brief period of **domain adaptation**; the following **linear growth** confirms **successful fine-tuning** on AICity traffic patterns.

Hyperparameters

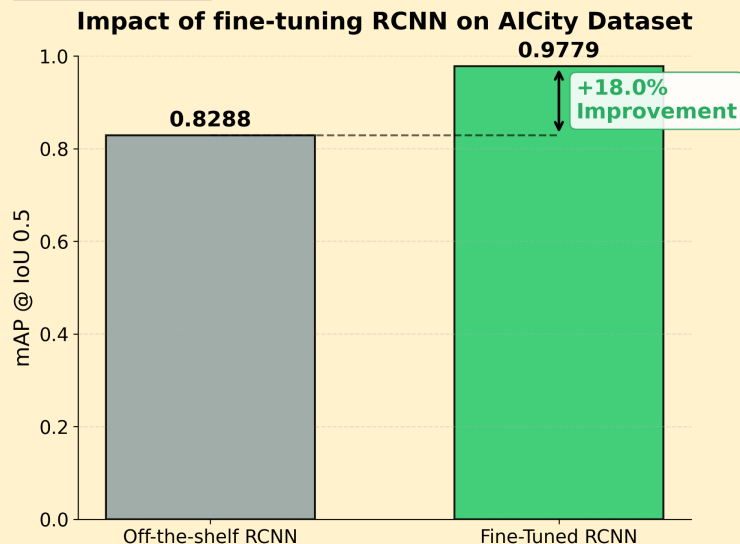
Fine-tuning based on [pytorch tutorial](#). The hyperparameters chosen to fine tune are:

- Optimizer: SGD
 - Learning rate: 0.005
 - Momentum: 0.9
 - Weight decay: 0.0005
- Scheduler: [StepLR](#)
- Batch size: 32 (max allowed by GPU)

Task 1.2: Object Detection: Fine-tune (Team 5)



Comparison

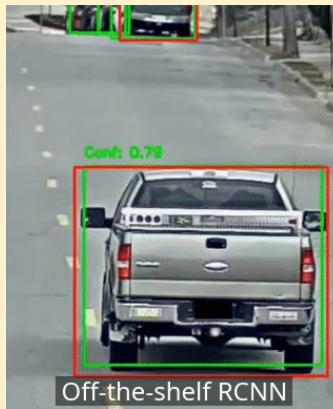


Fine-tuning significantly **boosts detection accuracy**, pushing performance very close to perfect detection (going from 0.82 to 0.97 mAP@0.5). We can deduce that the **pretrained** model **suffers** from **domain shift**, fine-tuning adapts feature representations to the AI City dataset and the detector becomes better calibrated to object scale, viewpoint, and scene characteristics. In the resulting video we can see fewer missed detections and better localization accuracy.

Qualitative results

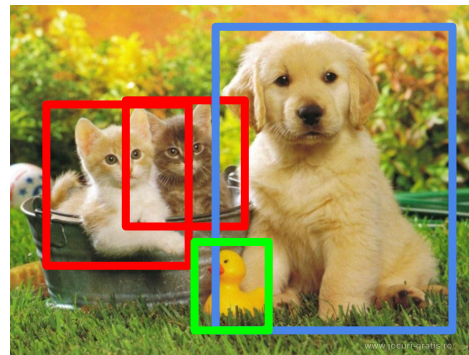
From the qualitative results, we observe that the **bounding box of the fine-tuned net remains stable** and consistent across frames. It does not disappear and reappear intermittently, which indicates improved temporal stability and more reliable tracking.

In contrast, the **off-the-shelf RCNN** exhibits noticeable **bounding box flickering** and unstable detections. Additionally, it detects background vehicles that are not annotated in the ground truth, introducing false positives. After fine-tuning, these spurious and unstable detections are largely eliminated, resulting in cleaner and more consistent predictions.



Tasks

- Task 1: Object detection
 - Task 1.1: Off-the-shelf
 - Task 1.2: Fine-tune to your data
 - **Task 1.3: K-Fold Cross-validation**
- Task 2: Object tracking



Task 1.3: K-Fold Cross-validation (Team 5)

Cross validation

To ensure we didn't get a "lucky" fine-tuning R-CNN with a specific set of frames (training with first 25% and testing with last 75% as in task 1.2), we implemented a k-fold cross validation.

Kfold allows having a more **robust evaluation** by using every sample as train and test. Thus, the resulting mean and variance provide more **statistically significant** basis for model selection and hyperparameter tuning.

Strategies

The strategies are performed with four folds (k=4).

Strategy A

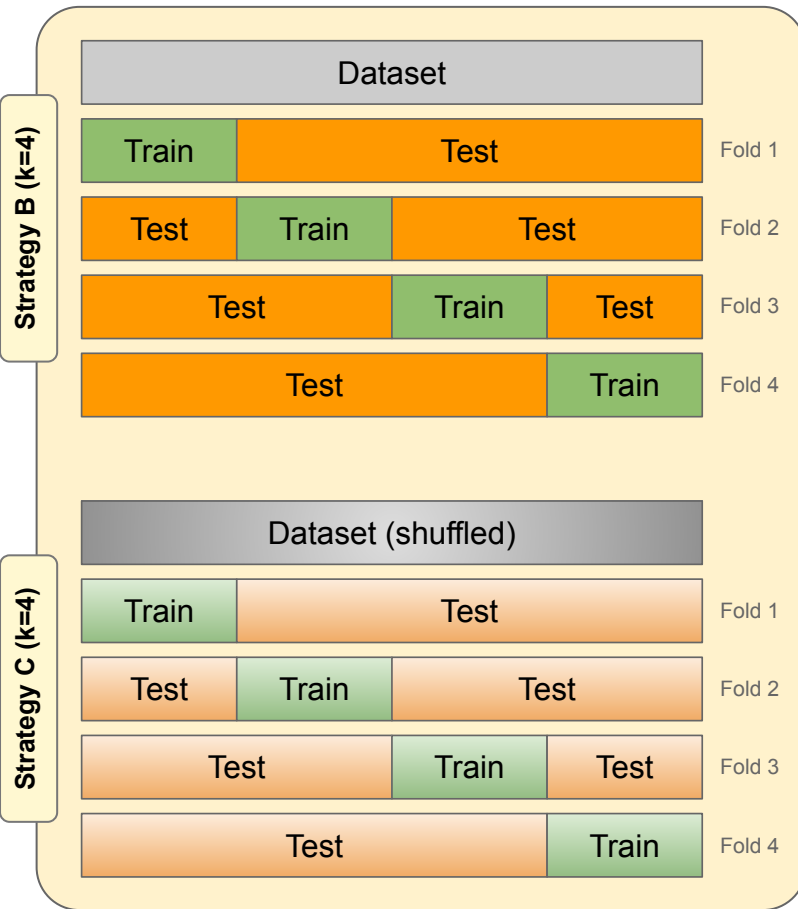
- First 25% for train last 75% for test.
- May have selection bias.

Strategy B

- Video split into 4 contiguous chunks. 3 for training, 1 for testing.
- Preserves temporal correlation.

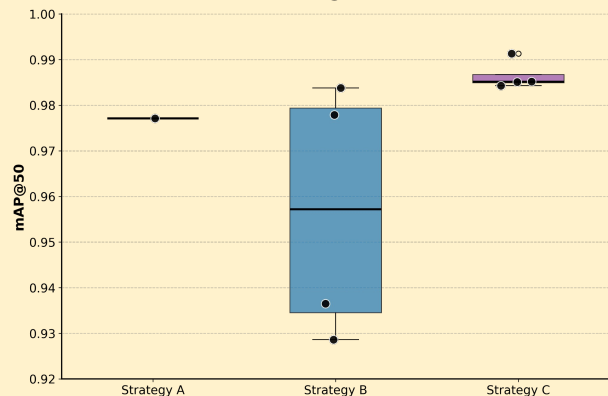
Strategy C

- Frames shuffled randomly.
- 25% random frames for train.
- Breaks temporal correlation.



Task 1.3: K-Fold Cross-validation (Team 5)

Cross-Validation Strategies Variance (mAP@50)



Strategy A: 0.9771 mAP@50

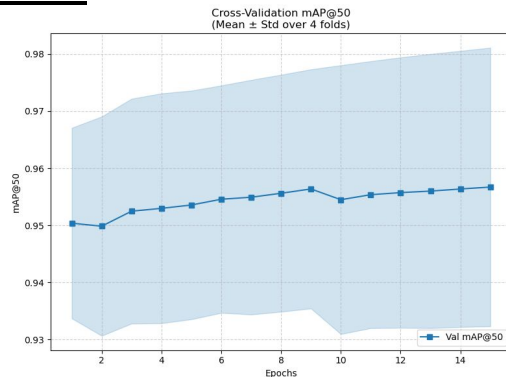
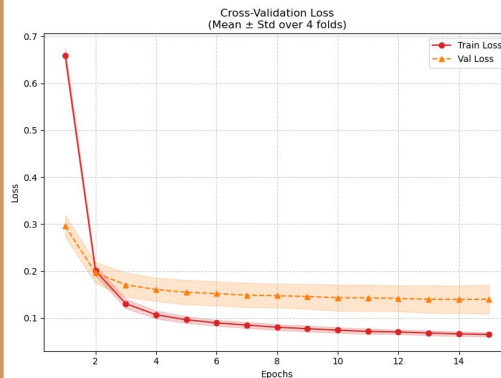
Strategy B: Mean mAP@50 = 0.9567 ± 0.0282

Strategy C: Mean mAP@50 = 0.9864 ± 0.0032

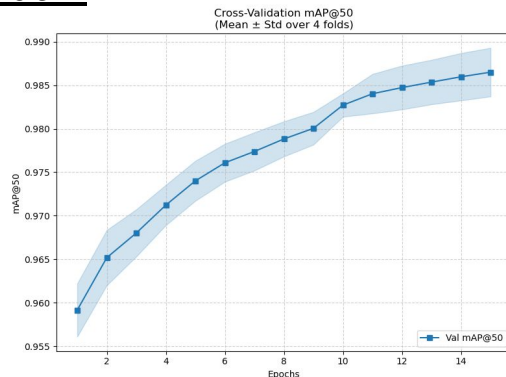
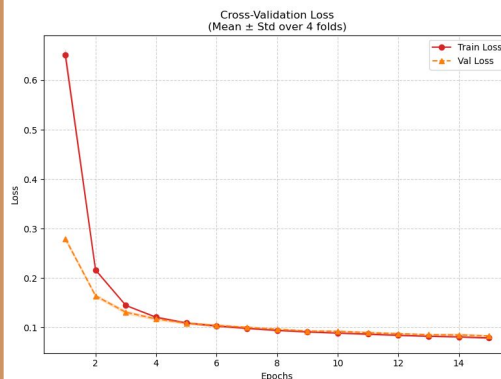
Strategy B: It has **lower mean performance**, **higher standard deviation**, its folds differ more strongly from each other. Strategy B splits the data in a way that exposes **variability across lighting conditions** or **traffic density**. This introduces **intra-video domain shift**. Therefore, performance varies more across folds.

Strategy C: Has **higher mean mAP** (0.98 mean mAP). It has very low standard deviation. Extremely **stable across folds**. Strategy C splits the dataset after being shuffled. Train and validation sets are very similar. This reduces distribution shift and makes the task easier. The model generalizes "within the same context" rather than across varied conditions.

Strat B



Strat C



Tasks

- Task 1: Object detection
- Task 2: Object tracking
 - **Task 2.1: Tracking by Overlap**
 - Task 2.2: Tracking with a Kalman Filter
 - Task 2.3: IDF1, HOTA scores



Task 2.1: Tracking by Maximum Overlap (Team 5)

Object Detection*

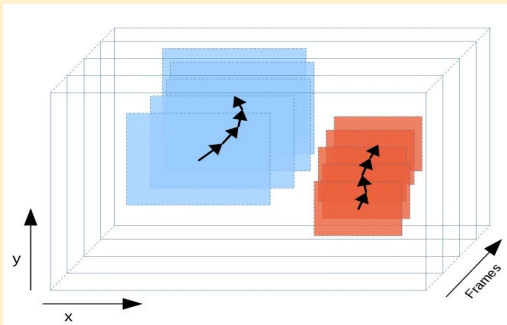
IoU Matrix

Hungarian Match

Update/Create/Delete
Tracks

Core logic

Assign IDs based on highest IoU overlap between Frame N and N+1.



Thresholds

Confidence threshold

- **Filters bounding boxes** with confidence lower than a given value.
 - High conf. thresh. minimize FP detections but risks losing tracks.

IoU threshold

- Defines a **minimum overlap** required to validate a match between an existing track and a new detection.
 - High IoU thresh. prevent ID swaps when we have dense traffic but fails if vehicles move fast between frames.

Max age

We implemented a max age logic for the IoU tracker:

- **Predict:** Every existing track starts the frame by incrementing its age.
- **Match:** If a detection overlaps with the track (High IoU), age is **reset to 0**.
- **Kill:** If age > max_age, the track is purged from the known tracks.

Higher max age handles longer occlusions but risks "drifting" (matching the wrong object later). Lower max age is more accurate but less robust to occlusions.

Metrics

To evaluate the tracking algorithms implemented we used the IDF1[1] metric and HOTA[1] metric.

We used the full video to evaluate the tracking algorithms.

[TrackEval](#) GitHub implementation used for the metrics.

Task 2.1: Tracking by Maximum Overlap (Team 5)

Parameter optimization

We performed a **bayesian search** to find the best parameters for IoU tracking.

Instead of choosing between maximizing **HOTA** or **IDF1**, we **maximized** both **simultaneously** using the following formula. By doing so, we aimed to achieve a balanced optimization that jointly improves detection accuracy and identity consistency, rather than favoring one aspect of tracking performance over the other.

$$\max \frac{1}{2} \text{HOTA} \times \frac{1}{2} \text{IDF1}$$

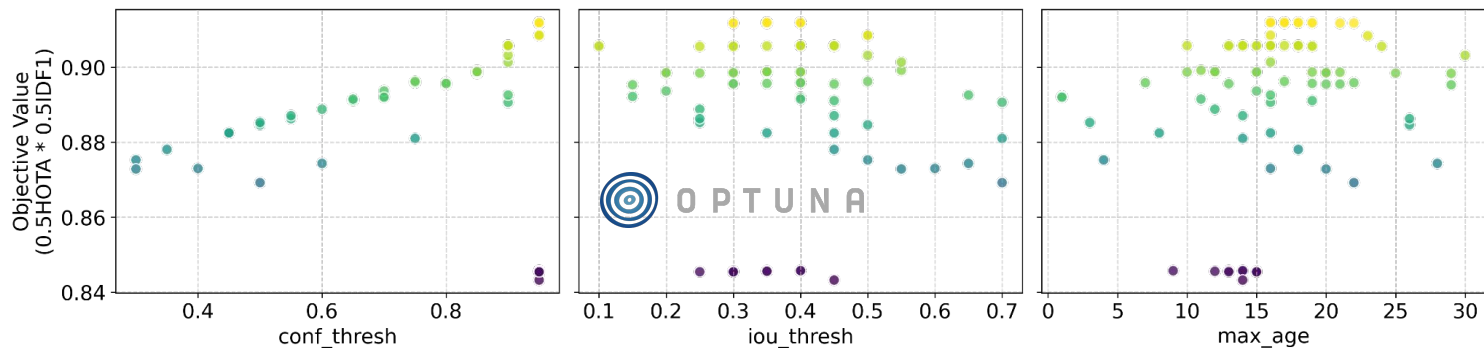
Search space

IoU threshold: [0.1 : 0.7]
Confidence threshold: [0.3: 0.95]
Max age: [0 : 30]



IoU thresh.	0.4
Conf. thresh.	0.95
Max age	16

Hyperparameter Slice Plot (IoU tracking)



Tasks

- Task 1: Object detection
- Task 2: Object tracking
 - Task 2.1: Tracking by overlap
 - **Task 2.2: Tracking with a Kalman Filter**
 - Task 2.3: IDF1 score



Task 2.2: Tracking with a Kalman Filter (Team 5)

Intuition

The Kalman Filter estimates the true state of a system by combining:

- A **motion model** (what we expect to happen)
- A **measurement** (what we observe)
- Their **respective uncertainties**

It continuously alternates between:

Predict → Update

Prediction Step

At time $k-1$, we know the estimated state:

$$\hat{x}_{k-1}, P_{k-1}$$

Using the motion model (in our case constant velocity), we predict the next state:

$$\begin{aligned}\hat{x}_k^- &= F\hat{x}_{k-1} \\ P_k^- &= FP_{k-1}F^T + Q\end{aligned}$$

Where:

- F → state transition matrix
- Q → process noise covariance
- $\hat{x}_k^-$ → predicted state (prior)
- P_k^- → predicted uncertainty

Measurement Step

We receive a new observation: z_k

Measurement model:

$$z_k = Hx_k + v_k$$

Where:

- H → observation matrix
- $v_k \sim N(0, R)$ → measurement noise
- R → measurement covariance

Update step

Now we combine prediction and measurement.

Compute Kalman Gain:

$$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$$

The Kalman Gain determines:

- If **measurement noise** is small → trust measurement more
- If **process noise** is small → trust prediction more

Update state estimate: $\hat{x}_k = \hat{x}_k^- + K_k (z_k - H\hat{x}_k^-)$

This corrects the prediction using the innovation (measurement error).

Update uncertainty: $P_k = (I - K_k H) P_k^-$

Uncertainty decreases after incorporating the measurement.

Numerical Stability Issue

While coding kalman we found that :

- The Kalman filter is purely linear.
- It does not enforce physical constraints.
- Predicted state can produce:
 - Negative area (s).
 - Negative aspect ratio (r).

Consequence

- IoU computation uses $\sqrt{s \cdot r}$.
- If s or $r < 0$ we get **NaN**.
- NaNs propagate to IoU matrix.
- Hungarian algorithm crashes.

FIX

Practical Fix

We enforce positivity:

if $s \leq 0$: $s = 1e-6$
if $r \leq 0$: $r = 1e-6$

With this fix we enforce a physical constraint, we prevent the NaN error and we ensure a stable assignment step.

Task 2.2: Tracking with a Kalman Filter (Team 5)

We performed a bayesian search to find the best params for Kalman filter tracking.



Search space

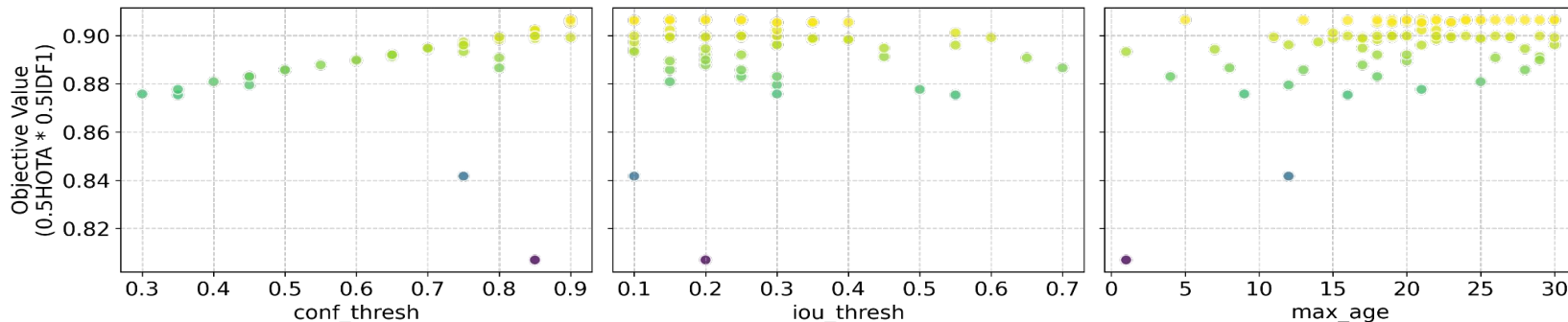
IoU threshold: [0.1 : 0.7]

Confidence threshold: [0.3 : 0.95]

Max age: [0 : 30]

Process noise: [0.01 : 100] (log scale)

Measurement noise: [0.01 : 100] (log scale)



Search analysis

Confidence Threshold (conf_thresh)

- Performance **increases** with **higher thresholds** because removing low-confidence detections reduces false positives. High detection confidence improves tracking robustness.

IoU Threshold (iou_thresh)

- Similar results between 0.1 and 0.4 IoU. It can be because overly strict matching harms association; too loose increases ID switches.

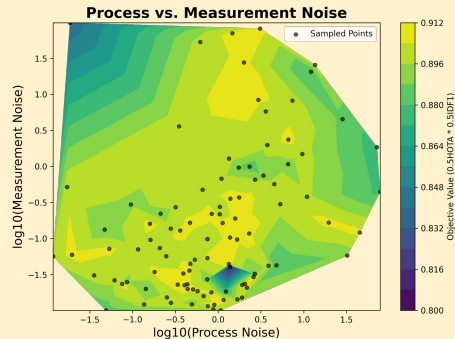
Max age (max_age)

- Allowing tracks to persist through short occlusions reduces ID switching or lost IDs. Track persistence improves IDF1.



IoU thresh.	Conf. thresh.	Max age	Process noise	Measure noise
0.20	0.90	17	1.33	0.04

Search analysis

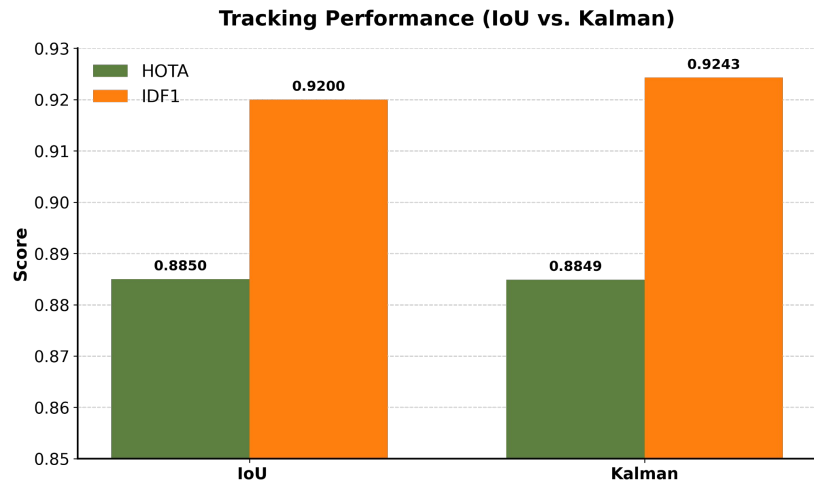


With this plot we compare the last two parameters: **Process noise** (model uncertainty) and **Measurement noise** (detection uncertainty).

Findings:

- Best performance found lies in a **balanced region**.
- Extremes in either direction degrade performance.
- The optimal values found by the search indicate that we must trust a bit more the process noise rather than the model and observations.

Task 2.2: Tracking with a Kalman Filter (Team 5)

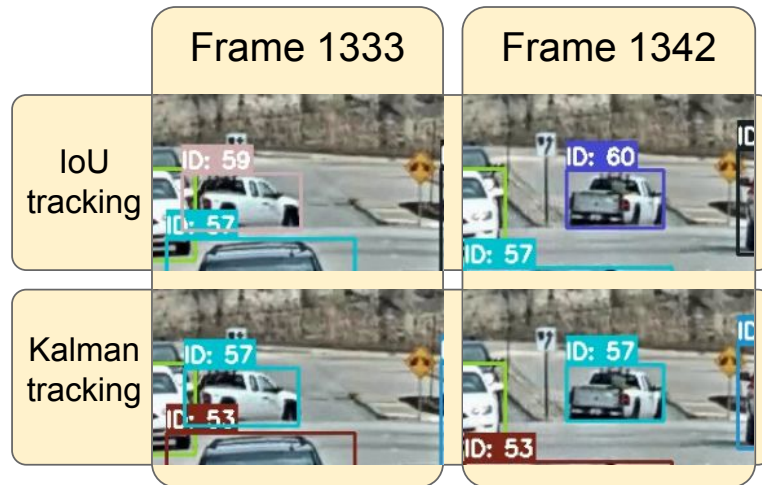


HOTA (0.8850 vs. 0.8849):

- Because both trackers get as input identical, almost perfect bounding boxes from the fine-tuned R-CNN, their foundational detection accuracy is exactly the same, resulting in a near-tie.

IDF1 Improvement (0.9200 → 0.9243):

- The Kalman Filter demonstrates a small advantage in identity preservation (IDF1).
- IoU tracker might drop an ID when spatial overlap falls to zero, the Kalman Filter's constant-velocity motion model successfully predicts the vehicle's location through the gap.



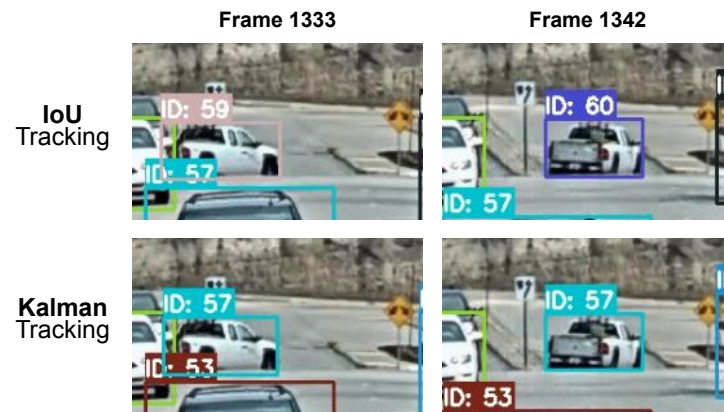
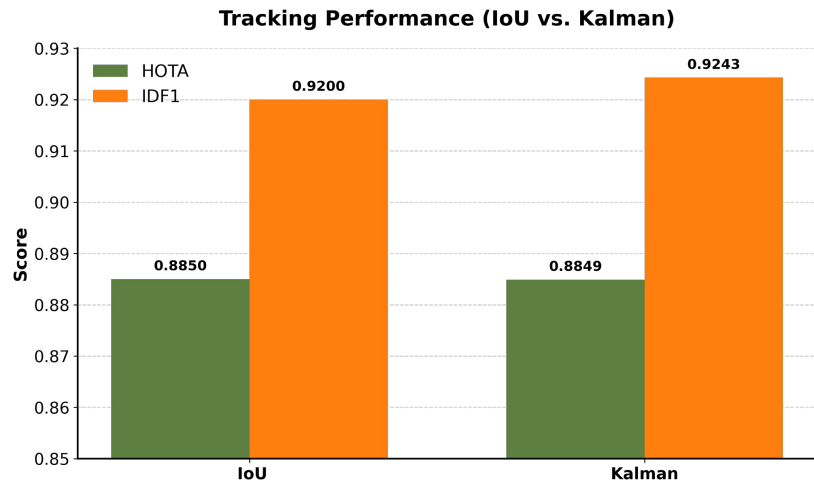
IoU (Top Row): Between frame 1333 and 1342, the IoU tracker suffers an ID switch on the white pickup truck (changing from ID 59 to ID 60).

- Because the truck is turning and altering its position and shape across this 9-frame gap, the spatial overlap between the last known bounding box and the new detection drops below the matching threshold, causing the tracker to lose the object and spawn a new ID.

Kalman Filter (Bottom Row):

- In the exact same scenario, the Kalman Filter successfully preserves the truck's identity (maintaining ID 57).

Task 2.2: Tracking with a Kalman Filter (Team 5)



HOTA (0.8850 vs. 0.8849):

- Because both trackers get as input identical, almost perfect bounding boxes from the fine-tuned R-CNN, their foundational detection accuracy is exactly the same, resulting in a near-tie.

IDF1 Improvement (0.9200 → 0.9243):

- The Kalman Filter demonstrates a small advantage in identity preservation (IDF1).
- IoU tracker might drop an ID when spatial overlap falls to zero, the Kalman Filter's constant-velocity motion model successfully predicts the vehicle's location through the gap.

IoU (Top Row): Between frame 1333 and 1342, the IoU tracker suffers an ID switch on the white pickup truck (changing from ID 59 to ID 60).

- Because the truck is turning and altering its position and shape across this 9-frame gap, the spatial overlap between the last known bounding box and the new detection drops below the matching threshold, causing the tracker to lose the object and spawn a new ID.

Kalman Filter (Bottom Row):

- In the exact same scenario, the Kalman Filter successfully preserves the truck's identity (maintaining ID 57).

Tasks

- Task 1: Object detection
- Task 2: Object tracking
 - Task 2.1: Tracking by overlap
 - Task 2.2: Tracking with a Kalman Filter
 - **Task 2.3: IDF1,HOTA scores**

